Project – Low Level Design

On

CI/CD Pipeline for Node.js Retail Application

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Amit Patidar	EN22CS301114
02	Aniket Kushwah	EN22CS301124
03	Anuj Singh Rathore	EN22CS301166
04	Anurag Didolkar	EN22CS301269
05	Arsh Patidar	EN22CS301204
06	Avani Gupta	EN22CS301236

Group Name: 09D2

Project Number: DO-21

Industry Mentor Name: Prof. Akshay Saxena University

Mentor Name:

Academic Year: 2025-2026

Table of Contents

1. Introduction.

    1.1. Scope of the Document
    1.2. Intended Audience
    1.3. System overview

2. System Design.

    2.1. Application Design
    2.2. Process Flow
    2.3. Information Flow
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue

3. References

4. GitHub

1. Introduction

This document presents the system design for the **CI/CD Pipeline for Node.js Retail Application** project, developed as part of the Datagami Skill Based Course at Medicaps University. The project demonstrates a complete DevOps implementation involving version control, containerization, automated build and deployment, and orchestration using modern tools.

The application is built using Node.js and is designed to simulate a retail system. The DevOps pipeline ensures that the application is automatically built, tested, containerized, and deployed with minimal manual intervention. This improves efficiency, reduces errors, and ensures faster delivery.

The system uses GitHub for version control, GitHub Actions (or Jenkins) for CI/CD automation, Docker for containerization, and Kubernetes for deployment and scaling of the application.

1.1 Scope of the Document

This document covers the complete low-level design of the CI/CD pipeline for the Node.js retail application. Specifically, it includes:

- Overall system architecture and workflow
- CI/CD pipeline design and automation process
- Docker containerization strategy
- Kubernetes deployment configuration
- Application structure and internal components
- Data flow and user interaction flow

1.2 Intended Audience

This document is intended for various stakeholders involved in the development, deployment, and evaluation of the CI/CD pipeline for the Node.js retail application.
The following audience can benefit from this document:

- **Developers**
  To understand the internal working of the application, code structure, and integration with the CI/CD pipeline.

- **DevOps Engineers**
  To understand the design and implementation of the CI/CD pipeline, containerization using Docker, and deployment using Kubernetes.
- **Project Evaluators**
  To assess the system design, workflow, and implementation of DevOps practices in the project.
- **Students and Learners**
  To gain knowledge about real-world DevOps implementation, including automation, continuous integration, and deployment strategies.

1.3 System Overview

The system is designed to implement a **CI/CD pipeline for a Node.js retail application**, which automates the complete software development lifecycle. The main objective of this system is to ensure continuous integration, continuous testing, and continuous deployment of the application with minimal manual effort.

In this system, developers write and update the application code and push it to a GitHub repository. Once the code is pushed, an automated CI/CD pipeline is triggered. The pipeline performs tasks such as installing dependencies, building the application, running tests, and creating a Docker image.

The Docker image is then pushed to a container registry and deployed using Kubernetes. Kubernetes manages the deployment, scaling, and availability of the application, ensuring that the system remains reliable and scalable.

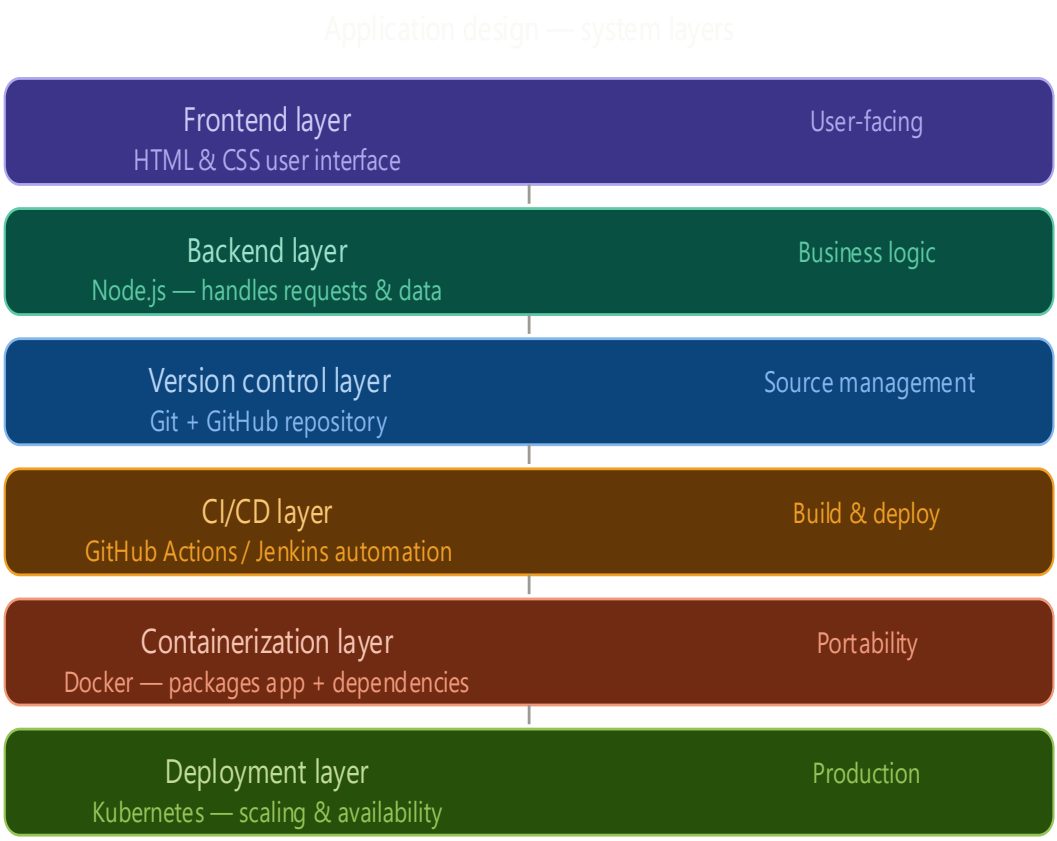
This system helps in reducing manual errors, improving deployment speed, and maintaining consistency across environments. It also ensures that the application is always up-to-date and available to users.

2. System Design

2.1 Application Design

The application is designed as a **Node.js-based retail system integrated with a CI/CD pipeline** to automate the build, test, and deployment processes.

The overall architecture of the system is divided into multiple layers:



- **Frontend Layer**
  The frontend is developed using HTML and CSS. It provides a simple user interface through which users can interact with the application.

- **Backend Layer**
  The backend is built using Node.js. It handles user requests, processes data, and sends responses back to the client.

- **Version Control Layer**
  Git is used for tracking changes in the source code, and GitHub is used as a remote repository for collaboration and storage.

- **CI/CD Layer**
  GitHub Actions (or Jenkins) is used to automate the process of building, testing, and deploying the application whenever changes are pushed to the repository.

- **Containerization Layer**
  Docker is used to package the application and its dependencies into containers, ensuring consistency across different environments.

- **Deployment Layer**
  Kubernetes is used to deploy, manage, and scale the containerized application, ensuring high availability and reliability.

This layered architecture ensures modularity, scalability, and efficient management of the application.

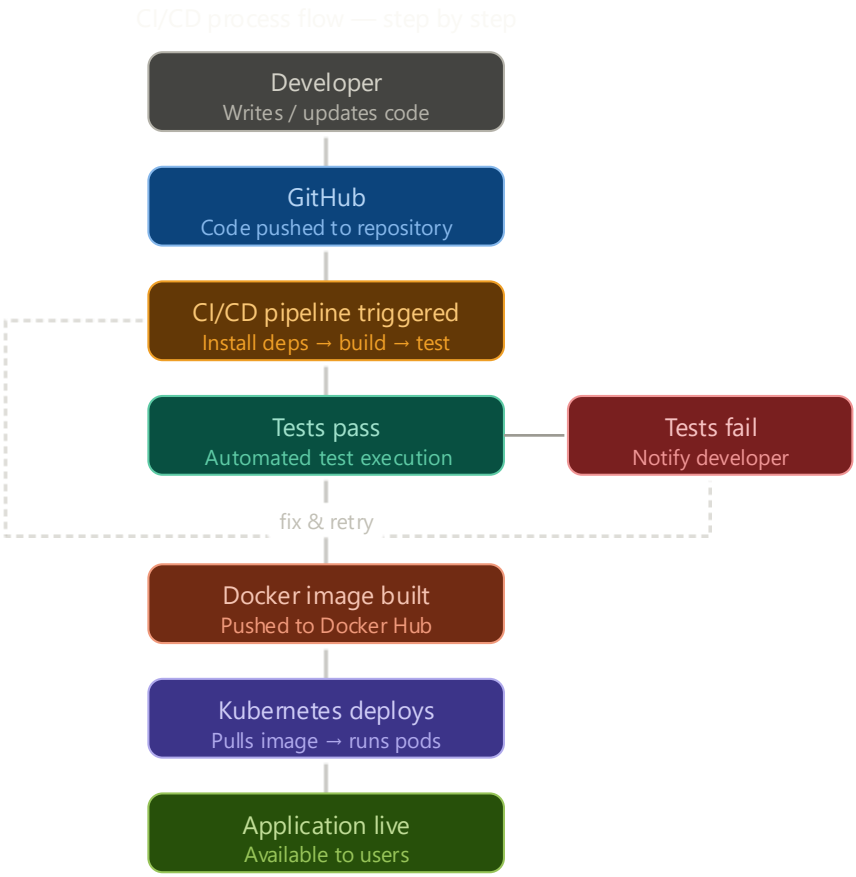
2.2 Process Flow

The process flow describes the step-by-step execution of the system from code development to application deployment.

The complete process is as follows:

1. The developer writes or updates the application code.
2. The code is pushed to the GitHub repository.
3. Once the code is pushed, the CI/CD pipeline is automatically triggered.
4. The pipeline installs the required dependencies and builds the application.
5. Automated tests are executed to ensure the application is working correctly.
6. A Docker image of the application is created.
7. The Docker image is pushed to a container registry (such as Docker Hub).
8. Kubernetes pulls the Docker image and deploys the application.
9. The application becomes available for users through a public endpoint.

This automated process ensures faster, reliable, and error-free deployment of the application.



2.3 Information Flow

The information flow describes how data moves through the system during both application usage and the DevOps pipeline process.

Application Information Flow:

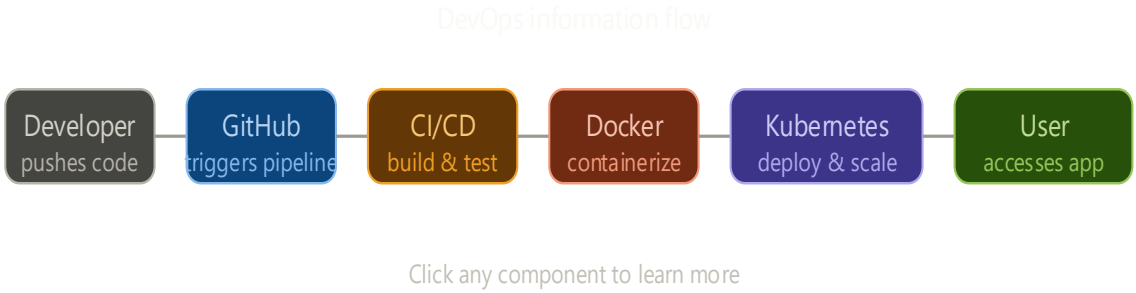
1. The user accesses the application through a web browser.
2. The browser sends a request to the Node.js server.
3. The server processes the request and performs the required operations.
4. The server sends a response back to the browser.
5. The result is displayed to the user.

Flow:
User → Browser → Server → Response → User

DevOps Information Flow:

1. The developer pushes code to the GitHub repository.
2. The CI/CD pipeline is triggered automatically.
3. The pipeline builds and tests the application.
4. A Docker image is created and stored in the container registry.
5. Kubernetes pulls the image and deploys it.
6. The deployed application becomes available to users.

Flow:
Developer → GitHub → CI/CD → Docker → Kubernetes → User



This structured flow ensures smooth communication between different components of the system and enables efficient automation.

2.4 Components Design

The system is divided into multiple components, each responsible for a specific functionality in the CI/CD pipeline and application deployment process.

Component 1- GitHub Repository

Property	Description
Platform	GitHub
Purpose	Stores application source code
Version Control	Tracks all code changes

Branching	Supports multiple branches for development
Collaboration	Enables team members to work together

GitHub is used as the central repository where developers push their code. It also acts as a trigger point for the CI/CD pipeline whenever changes are made.

Component 2- CI/CD Pipeline (GitHub Actions / Jenkins)

Step	**Action**	**Details**
1	Code Checkout	Fetch latest code from repository
2	Install Dependencies	Run `npm install`
3	Build Application	Prepare application for deployment
4	Test Execution	Run basic tests
5	Docker Build	Create Docker image
6	Push Image	Upload image to Docker Hub
7	Deployment	Deploy application to Kubernetes
8	Verification	Ensure deployment success

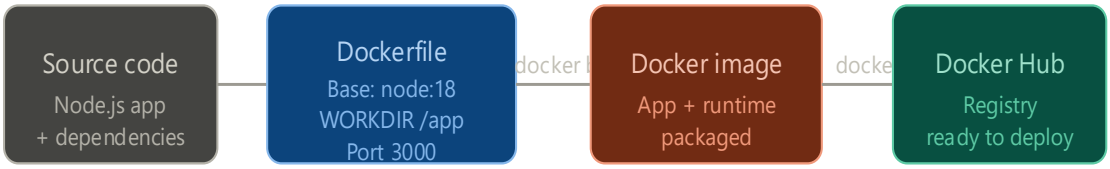
The CI/CD pipeline automates the entire process from code integration to deployment, reducing manual effort and ensuring faster delivery.

Component 3- Docker (Containerization)

Property	**Value**

Base Image	Node.js (node:18)
Working Directory	/app
Exposed Port	3000
Build Process	Copy code → Install dependencies → Run app
Purpose	Package application into container

Docker containerization— build process



Docker is used to containerize the application, ensuring that it runs consistently across different environments without dependency issues.

Component 4- Kubernetes Deployment

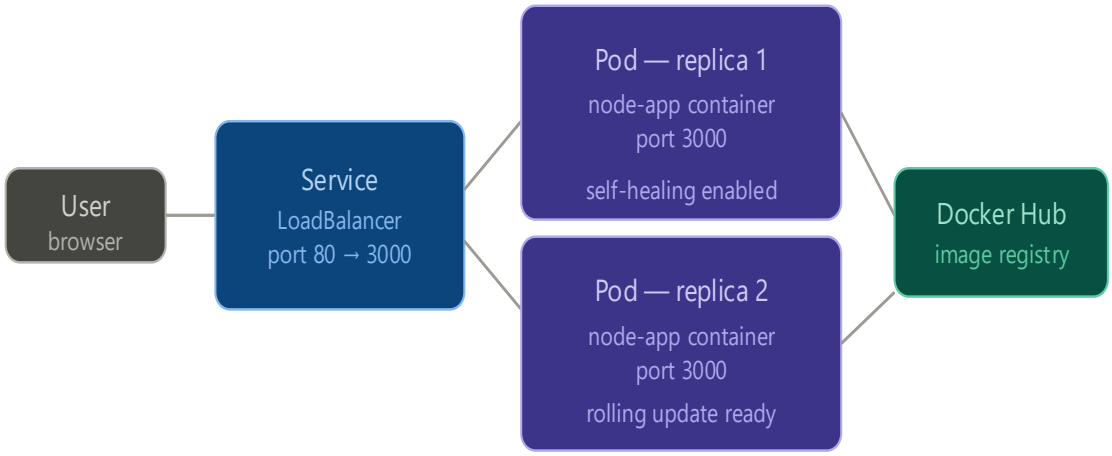
Parameter	**Value**	**Purpose**
Replicas	2	High availability
Strategy	Rolling Update	Zero downtime deployment
Container Port	3000	Application port
CPU/Memory	Limited	Resource management
Self-Healing	Enabled	Restart failed pods

Kubernetes manages the deployment of containers, ensuring scalability, load balancing, and automatic recovery in case of failures

Component 5- Kubernetes Service

Parameter	Value	Purpose
Service Type	LoadBalancer	Public access
Port	80	External access
Target Port	3000	Internal container port
Selector	app: node-app	Connects to pods

Kubernetes deployment architecture



2 replicas ensure high availability — if one pod fails, the other keeps running

The service exposes the application to users by routing external traffic to the correct Kubernetes pods.

2.5 Key Design Considerations

The following design considerations were taken into account while developing the CI/CD pipeline for the Node.js retail application:

1. Automation of Workflow

The entire process from code integration to deployment is automated using CI/CD tools. This reduces manual effort and minimizes human errors.

2. Scalability

Kubernetes is used to ensure that the application can scale horizontally by increasing or decreasing the number of pods based on demand.

3. High Availability

Multiple replicas of the application are deployed to ensure that the system remains available even if one instance fails.

4. Continuous Integration and Deployment

The system supports continuous integration and continuous deployment, allowing faster updates and quick delivery of new features.

5. Containerization

Docker is used to package the application along with its dependencies, ensuring consistency across development, testing, and production environments.

6. Version Control and Collaboration

Git and GitHub are used to manage source code, track changes, and enable collaboration among team members.

7. Reliability and Fault Tolerance

Kubernetes provides self-healing capabilities, automatically restarting failed containers and maintaining system stability.

8. Security Considerations

Sensitive information such as credentials and environment variables are managed securely using secrets and environment configurations.

2.6 API Catalogue

The application provides basic API endpoints to handle user requests and monitor the system. These APIs are used for communication between the client (browser) and the server (Node.js application).

Application API Endpoints

Endpoint	Method	Purpose
`/`	GET	Displays the home page of the application
`/products`	GET	Returns the list of available products
`/order`	POST	Handles order creation
`/health`	GET	Checks if the application is running properly

Health Check Endpoint

Endpoint	Method	Purpose
/health	GET	Returns status of the application (OK if running)

Response Example:

```
{
  "status": "OK"
}
```

External APIs (if applicable)

In this project, no external APIs are directly used. However, the system interacts with the following services during deployment:

- Docker Hub (for storing container images)

- Kubernetes API (for deployment and management)

3. References

The following resources were referred to during the development and implementation of the project:

- Official Node.js Documentation – https://nodejs.org
- Docker Documentation – https://docs.docker.com
- Kubernetes Documentation – https://kubernetes.io/docs
- Git Documentation – https://git-scm.com/docs
- GitHub Documentation – https://docs.github.com
- GitHub Actions Documentation – https://docs.github.com/en/actions

4. GitHub

https://github.com/ARSH2103/CI-CD-for-Node.js-Retail-App-.git